

Zephyr:
Clock Management Subsystem

Dylan Velasco
10.09.2026

restricted

Clock Management Subsystem

- Subsystem for managing clock tree reconfiguration, abstracting implementation details (static and dynamic).

- Peripherals request specific clock tree configuration/frequency via the framework.

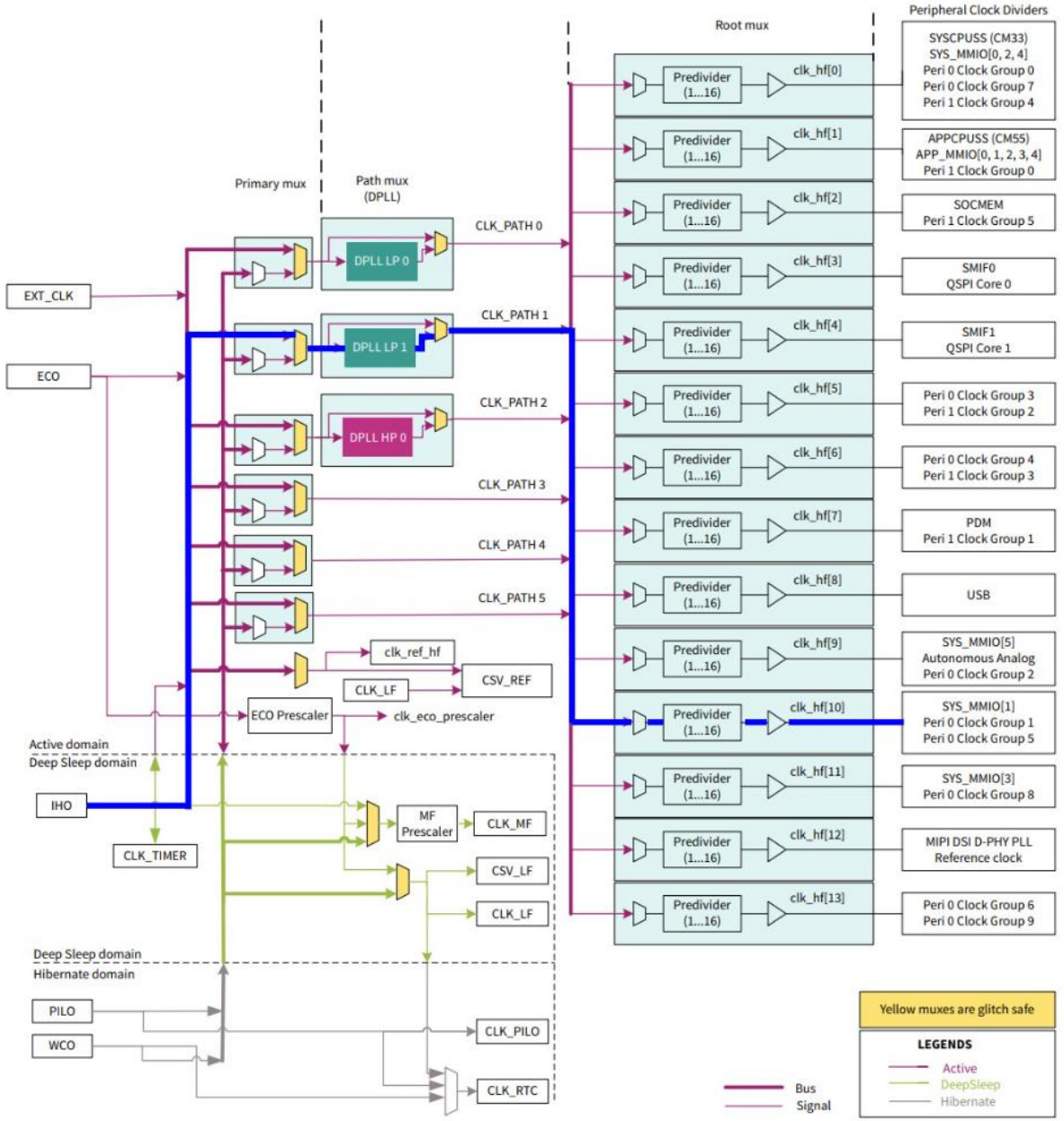
- States:
  - Specific configuration of a clock tree path.
  - Specific output frequency

- Requires knowledge of the SoC clock tree architecture

Copyright © Infineon Technologies AG 2023. All rights reserved.

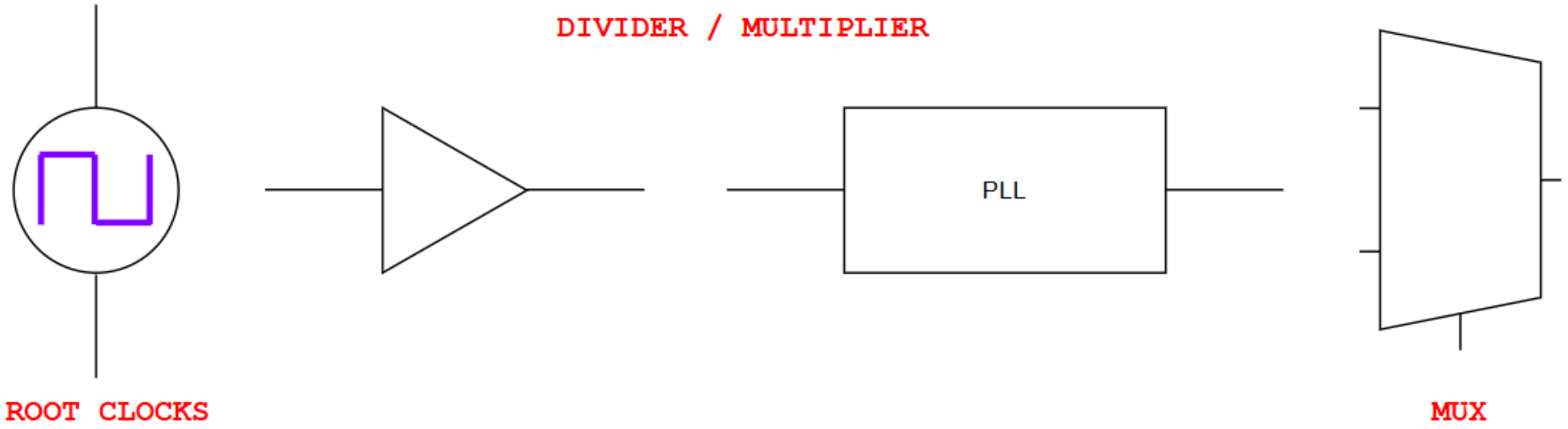
Clock Management Subsystem

- App developer defines the specific configuration of a clock path.
- As many configurations as needed.
- Described through Zephyr's devicetree.
- It creates a dependency chain.



Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree: Producers



– Any element whose main function is to take one or multiple input frequencies and produce an output frequency.

Copyright © Infineon Technologies AG 2023. All rights reserved.

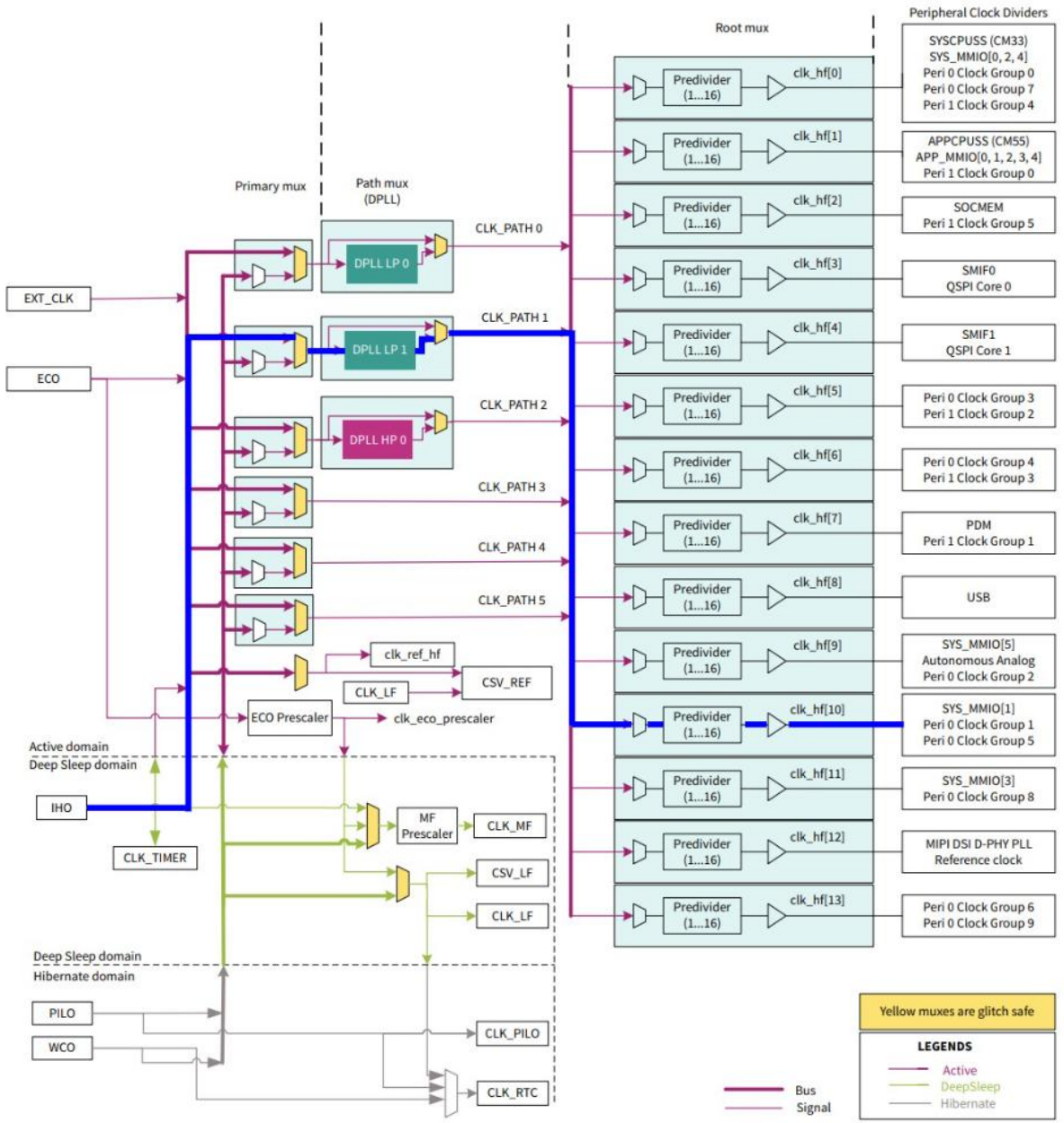
Clock Tree: Producers

- Each producer is represented as a device tree node.
- The clock tree is defined through references between producers.
- Each reference specifies the parent of a given producer.
- The complete clock tree topology is statically described at compile time.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree: Producers

```
clk_iho: clk_iho {
                clock-frequency = < 0x2faf080 >;
                system-clock = < 0x1 >;
                status = "okay";
                #clock-cells = < 0x1 >;
                compatible = "infineon,clock-root";
        };
path_mux1: path_mux1 {
                system-clock = < 0xb >;
                instance = < 0x1 >;
                compatible = "infineon,clock-mux";
                input-sources = < &clk_iho >;
        };
dpll_lp1: dpll_lp1 {
                compatible = "infineon,clock-pll";
                system-clock = < 0x18 >;
                reference-div = < 0x7 >;
                clock-frequency = < 0x17d78400 >;
                feedback-div = < 0x38 >;
                output-div = < 0x1 >;
                input = < &path_mux1 >;
        };
clk_hf10: clk_hf10 {
                system-clock = < 0x12 >;
                instance = < 0xa >;
                compatible = "infineon,clock-hf";
                clock-div = < 0x3 >;
                input-sources = < &dpll_lp1 >;
                };
```



Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree: Producers

The subsystem classifies each producer into one of three logical types based on its function:

1. Standard: Scales the input frequency by a configurable factor.
2. Mux: Selects one input signal from multiple available sources.
3. Root: Generates the base clock signal

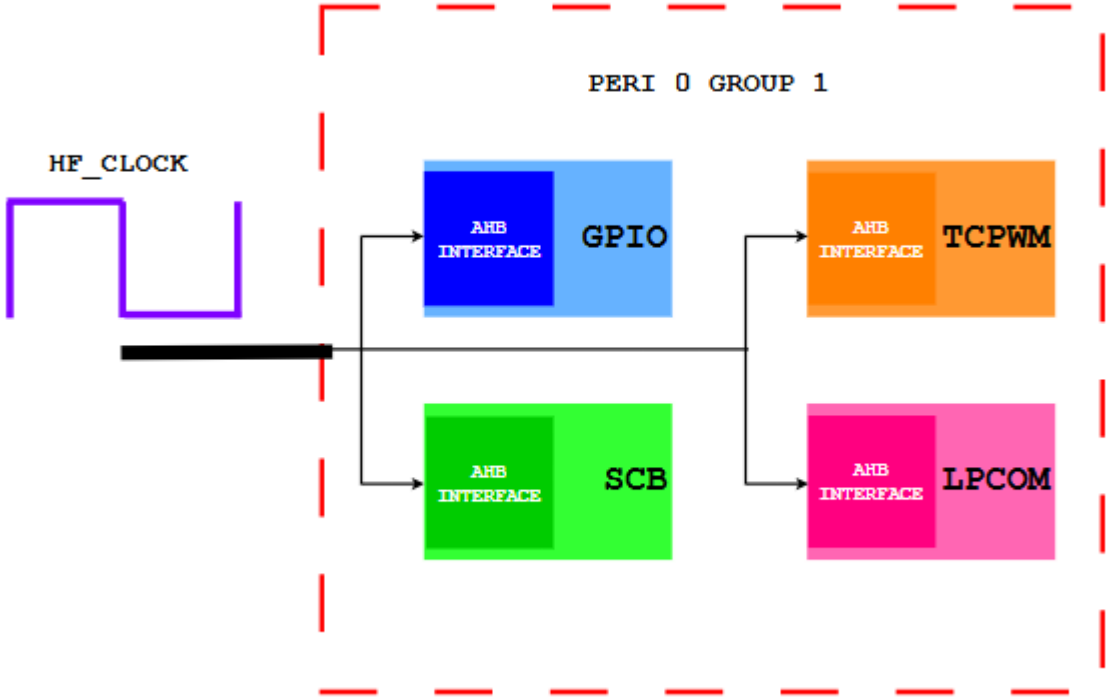
What does it mean?

- The clock tree is split into the smallest possible logical components.
- Each producer encapsulates a single operation and has its own driver.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree: Consumers

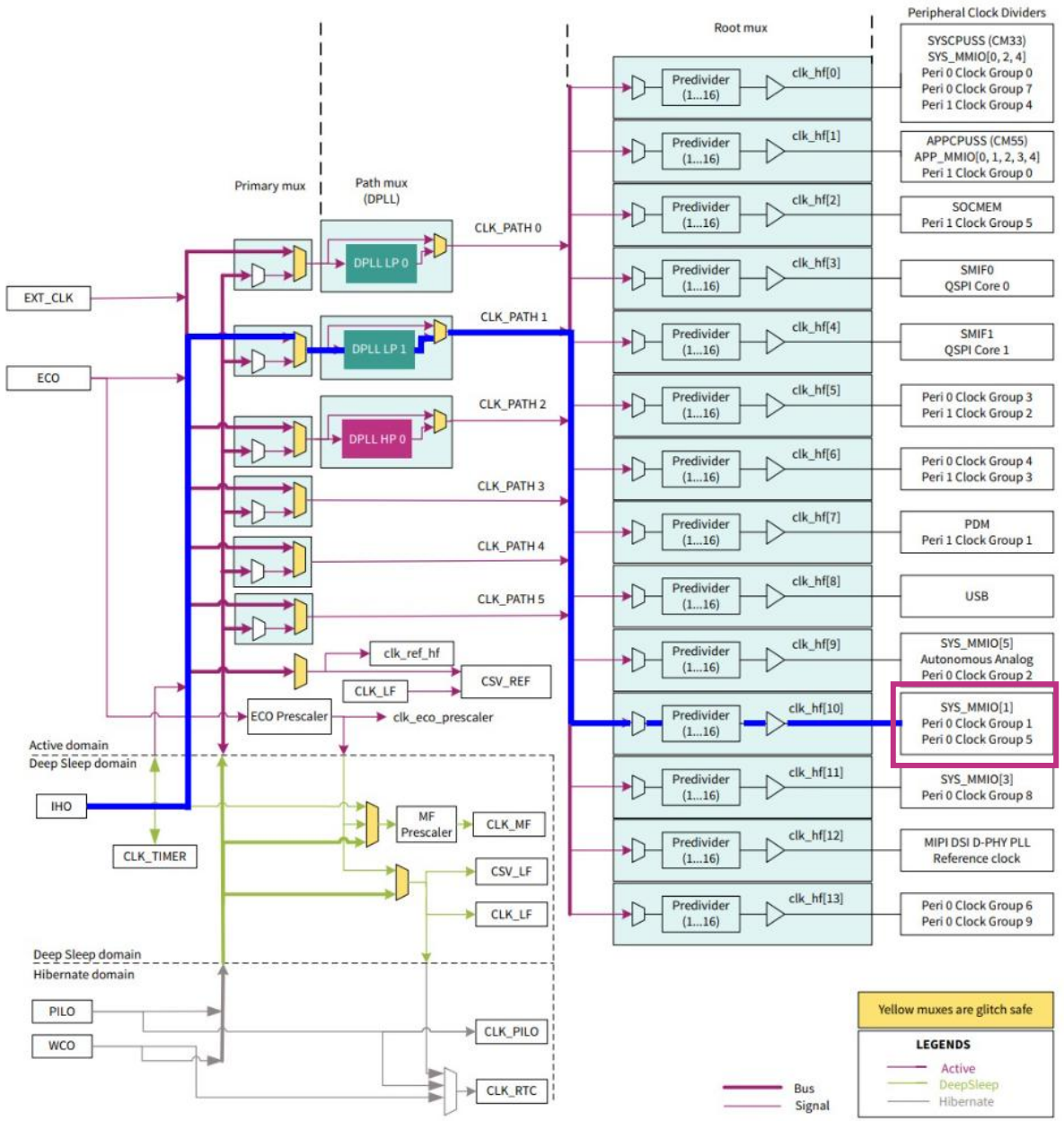
- Any element that uses the frequency produced by the clock tree.
- May request states ≈ specific operation frequencies.



Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree Consumers

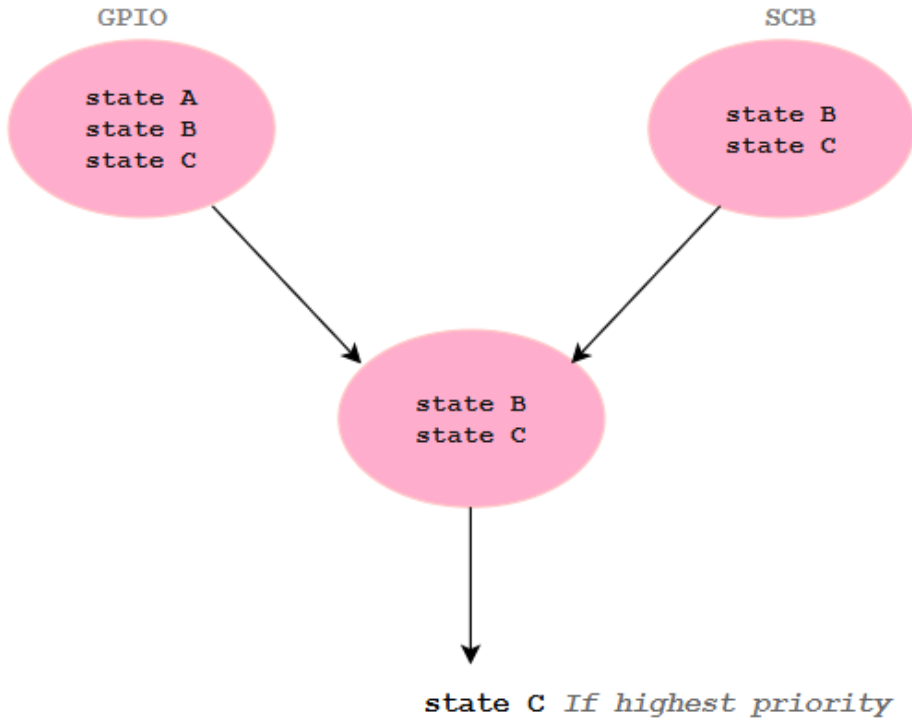
- SCB or GPIO peripherals request their operating frequency.
- The subsystem reconfigures the clock path.



Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree Consumers

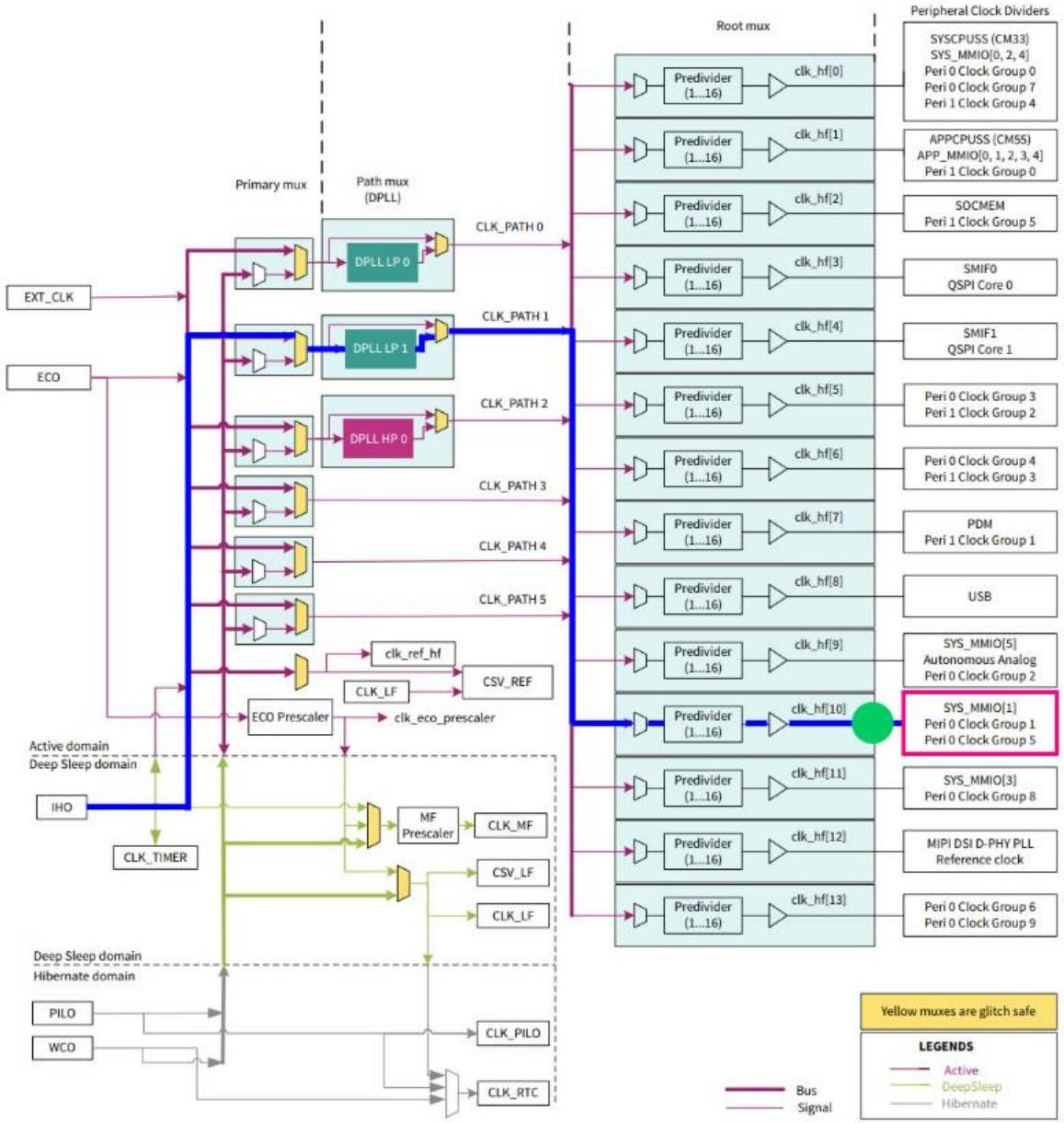
- Each consumer can request not just one state, but a set of states
- The subsystem intersects all requests and picks the highest-priority state
- If none matches all consumers, it returns a negative error code that must be handled by the developer



Copyright © Infineon Technologies AG 2023. All rights reserved.

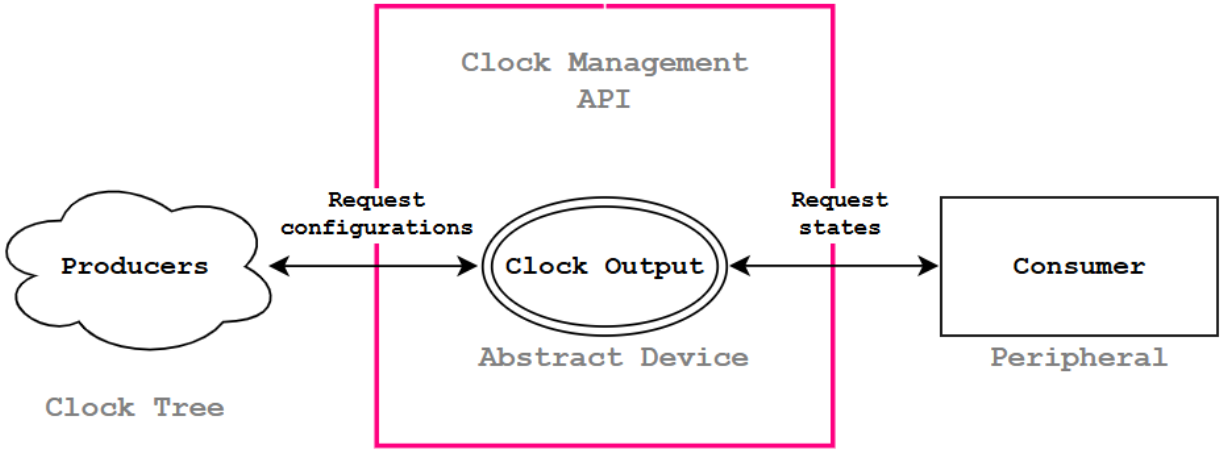
Clock Tree: Clock Output

- Represent logical connection points between producers and consumers.
- Define the states that consumers can request and producers can generate.
- They are the communication interface between both worlds.



Copyright © Infineon Technologies AG 2023. All rights reserved.

Framework Architecture



- Consumers request specific clock tree configurations.
- Reference a clock output node to identify the clock path to configure.
- The subsystem recursively applies configuration from nearest producer up to to the root.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree: Producer Driver

Producers are represented by a dedicated structure, struct clk:

- Hardware-specific data
- A reference to the clock's API
- Lists of the clock’s parents and children

Driver follows Zephyr’s style, but use _CLOCK_ macros instead of _DEVICE_ macros:

Has a smaller memory footprint.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree: Output Nodes

Must be defined as leaf nodes in the device tree:

- Child of the last producer node in the clock tree
- Its compatible needs to be '*clock-output*'.
- Its driver is implemented by the subsystem, since it's an abstract node.
- Clock consumers will reference these nodes to request states.

```
&clk_hf10 {
        clk_output_hf10: clk_output_hf10 {
        #clock-cells = <0>;
        compatible = "clock-output";
        };
};
```


Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree: Output Nodes

Each clock output defines a set of states:

- Different specific configurations of the clock path associated with that output.
- Consumers choose one or more of these available states to get a frequency.
- For many chosen states, the subsystem chooses the best according to a rank specifier.
- Producers within the clock path are reconfigured when a new state is selected

```
&clk_output_hf10: clk_output_hf10 {
        clk_output_hf10_default_state: clk_output_hf10_default_state {
            compatible = "clock-state";
            clocks = <&clk_iho 0x2faf080>, <&path_mux1 0>, <&dpll_lp1 1 0 56 7 1 1 0>, <&clk_hf10 0 3>;
            rank = <0>;
        };
};
```


Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree: Consumers

Consumers set request to get specific frequencies:

- Are a set of chosen available states in the clock output they belong to.
- Consumers choose one or more of these available states to get a frequency.
- The subsystem selects the lowest ranked state.

```
&scb2 {
        compatible = "infineon,scb_uart";
        clock-outputs = <&clk_output_hf10>;
        clock-output-names = "uart_clk";
        clock-request-0 = <&clk_output_hf10_default_state>;
        clock-request-names = "default";
};
```


Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree: Consumer Driver

It needs to define and initialize a **clock_management_data** structure:

- Reference to its clock output
- Clock requests declared by the consumer
- Stored within the device's config structure

This structure is then passed to the API functions to access the clock tree.

```
int clock_management_on(const struct clock_management_data *data, clock_output_t clk)
```


Copyright © Infineon Technologies AG 2023. All rights reserved.

Call Stack Example

```
Thread Name     Thread ID  CPU  Mode   Timestamp        Function(s)
-------------------------------------------------------------------------------------------------------------
      main   0x3403a240   0)    0 |     1829940 ns | instrumentation_trigger();                                  → User function
      main   0x3403a240   0)    0 |     1988990 ns | ifx_cat1_uart_clk_on() {                                    → User function
      main   0x3403a240   0)    0 |     2032660 ns |   clock_management_on() {                                   → Framework entry point function
      main   0x3403a240   0)    0 |     2078945 ns |     arch_irq_lock();
      main   0x3403a240   0)    0 |     2160865 ns |     clock_management_onoff() {                              → Main function that handles powering on and off of clock producers
      main   0x3403a240   0)    0 |     2205315 ns |       clock_onoff() {
      main   0x3403a240   0)    0 |     2251635 ns |         hf_clock_onoff() {
      main   0x3403a240   0)    0 |     2297555 ns |           Cy_SysClk_ClkHfEnable();                          → HF clock enable function
      main   0x3403a240   0)    0 |     2375765 ns |         };   /* hf_clock_onoff */
      main   0x3403a240   0)    0 |     2414060 ns |       };   /* lock_onoff */
      main   0x3403a240   0)    0 |     2456800 ns |       clock_get_type();                                     → Type of producer (Standard, root, mux)
      main   0x3403a240   0)    0 |     2533830 ns |       clock_get_type();
      main   0x3403a240   0)    0 |     2613145 ns |       clock_get_parent() {                                  → Gets parent of HF clock
      main   0x3403a240   0)    0 |     2654155 ns |         ifx_hf_get_parent();
      main   0x3403a240   0)    0 |     2731690 ns |       };   /* clock_get_parent */
      main   0x3403a240   0)    0 |     2775615 ns |       clock_onoff() {
      main   0x3403a240   0)    0 |     2820415 ns |         pll_on_off() {
      main   0x3403a240   0)    0 |     2867605 ns |           Cy_SysClk_DpllLpEnable();                         → PLL enable function
      main   0x3403a240   0)    0 |     2951550 ns |         };   /* pll_on_off */
      main   0x3403a240   0)    0 |     2988975 ns |       };   /* clock_onoff */
      main   0x3403a240   0)    0 |     3026700 ns |       clock_get_type();
      main   0x3403a240   0)    0 |     3101900 ns |       clock_get_type();
      main   0x3403a240   0)    0 |     3179540 ns |       clock_onoff() {
      main   0x3403a240   0)    0 |     3221935 ns |         mux_clock_onoff();
      main   0x3403a240   0)    0 |     3304170 ns |       };   /* clock_onoff */
      main   0x3403a240   0)    0 |     3343390 ns |       clock_get_type();
      main   0x3403a240   0)    0 |     3418595 ns |       clock_get_type();
      main   0x3403a240   0)    0 |     3493845 ns |       clock_get_parent() {                                  → Gets parent of PLL
      main   0x3403a240   0)    0 |     3534695 ns |         ifx_cat1_mux_get_parent() {
      main   0x3403a240   0)    0 |     3581245 ns |           Cy_SysClk_ClkPathGetSource();
      main   0x3403a240   0)    0 |     3662350 ns |         };   /* ifx_cat1_mux_get_parent */
      main   0x3403a240   0)    0 |     3702315 ns |       };   /* clock_get_parent */
      main   0x3403a240   0)    0 |     3740010 ns |       clock_onoff() {
      main   0x3403a240   0)    0 |     3785110 ns |         root_clock_onoff();                                 → Root's ON function
      main   0x3403a240   0)    0 |     3870970 ns |       };   /* clock_onoff */
      main   0x3403a240   0)    0 |     3910190 ns |       clock_get_type();
      main   0x3403a240   0)    0 |     3985060 ns |     };   /* clock_management_onoff */
      main   0x3403a240   0)    0 |     4023960 ns |     arch_irq_unlock();
      main   0x3403a240   0)    0 |     4099110 ns |   };   /* clock_management_on */
      main   0x3403a240   0)    0 |     4136345 ns | };   /* ifx_cat1_uart_clk_on */
      main   0x3403a240   0)    0 |     4178180 ns | instrumentation_stopper();
```


Copyright © Infineon Technologies AG 2023. All rights reserved.

Personal Feedback

- Simpler than other subsystems (e.g., Power Management).
- Powerful tool for static and dynamic reconfiguration of the clock tree.
- Still under development: documentation is not very clear and the subsystem is prone to changes.

Copyright © Infineon Technologies AG 2023. All rights reserved.


Infineon